

Rax: Strict Expressions for Verifiable Software Deployment

e. alvarez

ABSTRACT

Functional package management established that software deployment can be modeled as the evaluation of declared inputs into immutable results. Nix showed that isolated store paths, complete dependency closures, transactional profiles, and reachability-based garbage collection can make deployment safer and more flexible than destructive installation. Guix retained those mechanisms while using Scheme to describe packages and staged build programs. This paper proposes Rax, or Raku Expressions, as a third design point. Rax is a small external language implemented in Raku and compiled by Raku++. It evaluates expressions strictly and deterministically into a canonical plan intermediate representation (IR), while realizing only the requested plan and its dependency closure. Thus Rax is strict about meaning and lazy about work. The design separates a plan identifier (PlanId), which identifies a requested computation, from an object identifier (ObjectId), which identifies a realized file-system tree. An immutable realization record connects the two. Structured build operations, explicit capabilities, typed dependency classes, and atomic profile roots reduce the amount of policy hidden in shell scripts, environment variables, and mutable installation prefixes. The proposal preserves the central safety properties of functional deployment while narrowing the language and staging surface that must be understood, tested, and trusted. This paper describes the model, language boundary, store protocol, implementation strategy, and open research questions. Rax is presently a design and implementation program; empirical claims await a working prototype.

Introduction

Software deployment appears simple because its visible operation is often a copy: move a program from the place where it was built to the place where it will be used. The difficult part is not copying bytes. It is preserving the relationships that make those bytes meaningful. A program may depend on a particular compiler, library variant, build flag, interpreter, configuration, or generated file. A conventional installation can overwrite one of those relationships while leaving the dependent program in place. Upgrade and removal then become mutations of shared state whose consequences are only partly known.

Nix addressed this problem by separating user environments from installation space and by storing each component variant at a unique immutable path derived from its inputs [1]. Packages no longer overwrite one another, profiles can be switched atomically, and unreachable store objects can be collected safely. The store and its closures become a form of memory management for deployed software [2]. Guix, a package manager of the GNU Project, whose name is the recursive acronym "GNU's Not Unix," later retained the low-level functional deployment model while replacing the Nix expression language with Scheme interfaces and embedded domain-specific languages [3]. Its later g-expression mechanism made cross-stage references explicit and hygienic [4].

Rax begins from the judgment that these systems discovered the correct foundation, but not the final language boundary. Nix offers a small independent language whose pervasive laziness and string interpolation are powerful but can defer

errors and obscure staging. Guix gains the libraries, tooling, and extensibility of Scheme, but must manage a multi-tier programming environment in which host expressions generate build-side expressions. Rax proposes a narrower compromise: the implementation enjoys Raku as a general-purpose language, while package authors write a restricted external language that lowers to a small, canonical plan.

The design can be summarized in one sentence:

Rax is strict about meaning and lazy about work.

Expression evaluation is eager enough to produce a complete, inspectable plan before effects begin. Realization remains demand-driven: only the selected plan and its transitive closure are fetched or built. Strictness therefore does not mean building an entire package collection. It means that the meaning of the requested build contains no hidden language thunks when it is hashed, checked, transmitted, or authorized.

Rax is named for Raku Expressions. Its compiler and tools are written in Raku and target the Raku++ implementation. Raku++ is the normative implementation platform for the initial project. Compatibility with another Raku compiler is not a core conformance requirement, though contributors may propose it as an independent extension.

Motivation

The original Nix paper identified dependency completeness, variability, consistency, atomicity, exact identification, source and binary deployment, and the coexistence of central and local policy as the principal deployment problems [1].

Those problems remain central. Rax adds several language and supply-chain concerns that become visible after two decades of experience with functional package management.

Complete dependency descriptions. A build must not succeed because an undeclared compiler, shell, header, locale, or host tool happened to be present. Conversely, runtime dependencies must remain reachable after build inputs are no longer needed. A useful plan therefore distinguishes source, build, host, tool, and runtime inputs rather than placing every path into one untyped bag.

Exact variants. A package name and version are not sufficient identifiers. Patches, target platform, compiler, feature selection, builder, and environment all contribute to the requested computation. Rax therefore identifies a canonical build plan, not a human label.

Visible staging. Package systems necessarily cross stages. Source code is evaluated on the host; a builder later runs in an isolated environment; the result may run on yet another machine. Nix often crosses this boundary through strings and environment variables. Guix makes staged Scheme code first class through g-expressions [4]. Rax instead lowers the host expression to a closed, data-only plan before the build stage begins.

Reproducibility as evidence, not assumption. A plan can be deterministic while its builder is not. Time, random seeds, directory order, compiler nondeterminism, and undeclared kernel behavior can produce different bytes from the same declared inputs. Rax must distinguish the identity of the requested computation from the identity of an observed result. A build that disagrees with a prior result should create evidence of non-reproducibility rather than silently overwrite history.

Auditable effects. The evaluator should not perform arbitrary network, process, clock, or host-file-system operations. Fetching and building are explicit effects mediated by the realizer. The question "what can this build do?" should be answerable by inspecting its plan and granted capabilities.

Portable implementation. Rax should not require changes to the Raku++ parser or runtime before it can prove the deployment model. Raku grammars and action objects are designed to parse structured languages and construct native data [7]. Raku multi-dispatch provides a direct way to implement evaluators, canonical encoders, and validators over a small algebra of values [8]. Raku++ already supplies a hand-written front end, tree evaluator, and native C++ compilation path for the supported Raku language [9].

Design Goals

Rax has eight primary goals.

1. Concurrent versions and variants must coexist without overwriting one another.
2. A successful evaluation must produce a complete, canonical plan before any fetch or build effect begins.
3. The requested computation and the realized content must have separate, verifiable identities.
4. Profiles, upgrades, downgrades, and rollbacks must be atomic root changes.
5. Garbage collection must operate on explicit reachability and must explain why an object is retained.
6. Build actions and capabilities must be represented as structured data by default; shell programs remain an explicit escape hatch.
7. Source builds and binary substitution must share one realization protocol.
8. The language core, canonical representation, and store protocol must remain small enough to audit and reimplement.

Rax is not initially intended to be an operating-system configuration language, a service manager, a package collection, or a universal build system. Those layers may be built above the deployment core after its identities and invariants are stable.

Overview

The Rax system contains five principal parts: a source language, a pure evaluator, a canonical Plan IR, a realizer, and an immutable store. Profiles provide user-visible environments, and caches provide alternate realizations. Figure 1 shows the compilation path.

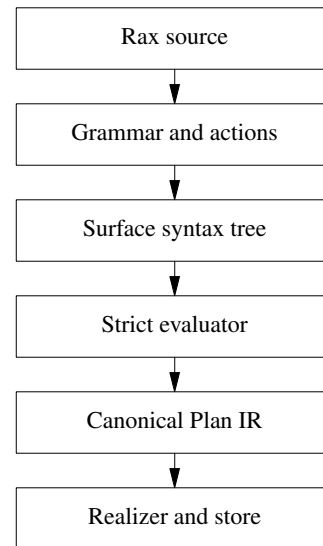


Figure 1: The Rax compilation and realization boundary.

The boundary between the evaluator and the realizer is the architectural center. Everything above that boundary computes meaning. Everything below it performs authorized effects. The Plan IR can be printed, hashed, signed, transmitted, compared, and checked without retaining parser or evaluator state.

The system is demand-driven at two levels. First, an import graph need only load modules required to resolve the selected export. Second, the realizer only builds the selected plan and dependencies reachable from it. No unrelated package is built merely because it was present in the same repository.

Rax Expressions

Rax source resembles Raku where Raku's native notation improves clarity, but it is not unrestricted Raku and is not evaluated with the Raku runtime-evaluation routine (EVAL). A dedicated grammar parses a deliberately small language. The initial value algebra contains the categories in Table 1.

Category	Values
Scalar	Nil, Bool, Int, Rat, Str, Path
Collection	List, Map, Set
Deployment	Source, Input, Output, Step, Plan

Table 1: Initial Rax value algebra.

The initial expression forms are literals, lists, maps, lexical bindings, pure functions, calls, named arguments, conditionals, assertions, imports, and field selection. Mutation, classes, arbitrary methods, user-defined operators, macros, slangs, runtime evaluation, ambient module loading, and implicit infinite sequences are excluded from the language core.

An illustrative package expression follows. Its fixed source is described by a Uniform Resource Locator (URL) using Hypertext Transfer Protocol Secure (HTTPS) and by a digest from the 256-bit Secure Hash Algorithm (SHA-256). The precise surface syntax may change during implementation; the semantic shape is the important part.

```
let mk-hello = --> :$tests = False {
  package(
    name => "hello",
    version => "2.12.1",

    source => fetch(
      url => "https://example.invalid/hello.tar.gz",
      sha256 => "...",
    ),

    inputs => {
      build => [$cc, $make],
      runtime => [$libc],
    },

    steps => [
      exec($src.child("configure"),
        arg("--prefix=", $out)),
      exec($make),
      $tests ?? exec($make, "check") !! skip(),
      exec($make, "install"),
    ],
  )
};

export default = mk-hello();
```

Functions describe variants by returning new values. They do not modify an existing package object. This preserves the useful functional composition found in Nix and Guix without requiring a class hierarchy for record-shaped data.

Imports are pure source dependencies. Their contents, module identities, and resolution rules contribute to the PlanId. Import resolution may be memoized, but memoization cannot

change observable results. A lock file may record resolved source revisions; it is an input to evaluation, not a second package language.

Strict Evaluation and Demand-Driven Realization

Nix's language is purely functional, lazy, and dynamically typed [3]. Laziness allows large recursive package sets and avoids evaluating values that are never demanded. It also means that an error can remain dormant until a distant field is forced, and that canonical export of an expression may require forcing more of the graph than the consumer expected. Guix uses an ordinarily strict Scheme host but introduces selected thunked fields and explicit staged expressions where delayed computation is useful [3,4].

Rax adopts strict evaluation as the default semantic rule. A selected export is evaluated to normal form before plan validation and hashing. The result may contain references to other Plans, Sources, and Steps, but it contains no unevaluated Rax expression. This gives four practical properties.

First, errors appear during evaluation, with source spans that identify the expression that produced them.

Second, canonicalization operates on values rather than thunks, closures, or evaluator state.

Third, policy checks can inspect the whole plan before authorizing effects.

Fourth, remote builders need only implement the Plan IR and store protocol, not the Rax surface language.

Strictness does not imply eager realization. A plan is a description of work, not the work itself. The command:

```
rax build packages.rax#hello
```

may evaluate the selected export and its semantic dependencies, query caches, and realize only the closure needed for hello. Documentation, tests, or other exports are not realized unless reachable. If real package definitions later show that explicit delayed values are necessary, Rax can add a named delay construct with defined forcing and hashing semantics. Pervasive implicit laziness is not part of the initial trusted core.

Canonical Plan IR

The evaluator lowers surface values to a smaller store language called the Plan IR. This follows the compiler separation already present in the original Nix design, where high-level expressions are translated to simpler store expressions [1]. The Plan IR describes one or more outputs and contains only canonical data.

A Plan contains at least:

- a format version and digest algorithm;
- a target and, where relevant, build platform;
- a set of named outputs;
- classified inputs and their exact plan or source identities;
- a sequence or graph of structured build Steps;

- a normalized environment map;
- declared capabilities;
- runtime reference expectations; and
- metadata that affects realization.

Human metadata such as a synopsis may accompany a Plan, but fields that do not affect realization must be explicitly excluded from PlanId computation. This prevents a spelling correction from rebuilding software while preventing a build-relevant flag from escaping the identity.

The Rax Canonical Encoding, version 1 (RAX-CANON/1), is independent of native hash iteration order and source formatting. It uses explicit type tags, strings encoded with the 8-bit Unicode Transformation Format (UTF-8) and prefixed by byte length, normalized integers and rational numbers, ordered lists, map keys sorted by encoded bytes, sets sorted by encoded element bytes, and versioned path rules. Floating-point values, timestamps, native object identities, and host-dependent textual renderings are forbidden in Plans.

The canonical encoder is part of the compatibility contract. A newer encoding uses a new version tag rather than silently changing old PlanIds.

Dual Identity and Realization Records

A functional deployment system must identify both a requested computation and an observed result. Let H denote the selected cryptographic hash function. The Rax Tree Encoding, version 1 (RAX-TREE/1), canonically represents a realized file-system tree. Rax defines the identities mathematically as follows:

$$\text{PlanId} = H(\text{RAX-CANON/1}(\text{Plan}))$$

$$\text{ObjectId} = H(\text{RAX-TREE/1}(\text{realized tree}))$$

Identifier	Meaning
PlanId	Requested computation
ObjectId	Realized output tree

Table 2: The two Rax identities.

A PlanId is known before building. It identifies the builder, declared inputs, platform, environment, steps, outputs, and capabilities. It is the key used to ask whether a suitable realization already exists.

An ObjectId is known only after realization. It identifies a normalized output tree: file contents, relative paths, executable bits, symbolic-link targets, and other explicitly supported metadata. Ownership, timestamps, and incidental file-system allocation are normalized or excluded.

An immutable realization record connects them:

(PlanId, platform, output-name)
-> (ObjectId, references, metadata, attestations)

This separation makes non-reproducibility visible. If the same PlanId is built twice and yields different ObjectIds, the store records a conflict or multiple observations; it never silently declares the results identical. Conversely, different Plans that produce the same normalized content can share physical storage when their runtime addressing permits it.

A difficulty arises because many builds embed their own installation prefix. The final content identity is unknown before the build, but the prefix must be known while configuring and linking. Rax therefore separates logical plan addresses from physical object addresses.

```
/rax/store/plans/<PlanId>/<output>
-> /rax/store/objects/<ObjectId>
```

The logical plan address is known before realization and is the path exposed to the builder. After success, an immutable realization binds that address to an object. Programs may retain plan addresses without creating a circular hash over their own ObjectId. If two plans produce byte-identical trees that embed different plan addresses, their ObjectIds will correctly differ. If they do not embed those addresses, the underlying object may be shared.

A PlanId and ObjectId include their algorithm names. For example:

```
plan:sha256:4c8d...
object:sha256:a891...
```

Algorithm agility is thereby represented in data rather than hidden in a fixed path convention.

The Store

The store contains immutable source objects, realized objects, plans, realization records, profiles, roots, leases, locks, and private temporary build directories. The Rax store-root environment variable (RAX_STORE) names its root. A possible layout is:

```
$RAX_STORE/
objects/<algorithm>/<ObjectId>/
plans/<algorithm>/<PlanId>.raxp
realizations/<PlanId>/<platform>/<output>.raxr
aliases/<PlanId>/<output>
profiles/<name>/generations/
profiles/<name>/current
roots/
leases/
locks/
tmp/
```

A store commit follows a transaction-like protocol.

1. Acquire a lock for the requested plan output.
2. Realize all required input closures.
3. Create a private build directory and normalized environment.
4. Execute the authorized builder steps.
5. Normalize and scan each declared output.
6. Compute its ObjectId and verify declared references.
7. Atomically commit the object and realization record.
8. Create the immutable logical alias and release the lock.

Objects are never modified after commit. A failed build leaves no visible partial object. Temporary paths are either removed or retained under an explicit debugging policy outside the object namespace.

Dependencies and Closures

Rax classifies dependencies because build-time presence and runtime reachability are different questions.

source	A fixed source tree, archive, generated source, or local immutable input.
build	An input needed to execute the build but not expected at runtime.
tool	A helper executable used by structured Steps.
host	A library or component linked or loaded for the target program.
runtime	An input that must remain in the realized runtime closure.

The Plan is authoritative. Output scanning is a verifier, not the primary source of dependency semantics. The scanner searches files and links for known logical plan addresses and platform-specific binary references. The outcomes are:

- a declared runtime reference is found: valid;
- a declared runtime reference is not retained: warning or optimization opportunity;
- an undeclared retained store reference is found: error in strict mode; and
- an external absolute dependency is found: error unless a declared capability permits it.

Some dependencies are semantic but not embedded as bytes, such as plugins, data files, interpreters selected by a launcher, or resources discovered from a manifest. These remain explicitly declared runtime references. Garbage collection traces both declared references and verified retained references.

A deployment closure is the transitive set of realized objects reachable from a selected realization. Copying a closure to another store is complete when all realization records, objects, and logical aliases required by that closure are present.

Structured Builders and Capabilities

The early Nix design passed derivation attributes to builders through environment variables and commonly delegated the build to shell scripts [1]. Guix demonstrated that build programs can be written in Scheme, improving composition and reducing dependence on shell text [3]. Rax adopts a data-first builder interface.

The standard Step algebra initially includes:

```
exec mkdir copy move remove
write symlink unpack patch
```

An exec Step contains an exact executable input, an argument vector, working directory, environment delta, expected exit behavior, declared output access, and capability requirements. It does not reconstruct a command by concatenating unquoted strings. Paths and outputs are typed values rather than accidental shell syntax.

A shell Step remains available because many upstream build systems expose only a shell-oriented interface. It is an explicit escape hatch whose interpreter, script bytes, environment, and capabilities all contribute to the PlanId. Tools may warn

that a shell Step is less analyzable than structured Steps, but they do not pretend that the escape hatch does not exist.

The realizer grants capabilities rather than assuming perfect isolation. A Plan may request:

- read access to declared inputs;
- write access to declared outputs and temporary storage;
- process execution of declared tools;
- network access for a fixed-output fetcher;
- specific devices or kernel facilities; or
- clock, randomness, locale, or user-database access.

The default build receives no network, home directory, ambient executable search path (PATH), host package database, or arbitrary device access. Platform back ends may provide different strengths of enforcement. Rax distinguishes three claims:

normalized	a clean environment and explicit inputs;
isolated	process and file-system isolation; and
hermetic	isolation plus the exclusion or control of non-deterministic ambient capabilities.

A platform must not call a build hermetic merely because it cleared environment variables.

Sources and Fetchers

Evaluation does not contact the network. A Source value describes an expected immutable input. A fetcher is a specialized realization whose network access is explicit and whose result is accepted only if it matches a declared content identity.

Initial source forms may include HTTPS archives, Git commits, and local trees. A moving branch name or unverified URL cannot directly enter a reproducible Plan. Resolution tools may discover a revision and update a lock file, but the resolved revision and content digest become the Plan inputs.

Secrets are capabilities, not plan values. Credentials must not appear in canonical Plan IR, logs, store paths, or cache records. A private fetch may request a named secret channel whose existence and policy are outside the content identity; the fetched result is still checked against its declared content digest.

Profiles, Generations, and Atomicity

A profile is an immutable manifest selecting realized outputs and defining collision policy for their user-visible names. Building a new profile creates a new generation. Activation changes one root, normally by atomic rename of a symbolic link:

```
profiles/work/current -> generations/43
```

Users and processes therefore observe either the old generation or the new one, not a mixture. Rollback switches the root to an earlier generation. The store objects remain untouched.

Profiles may be personal, site-wide, task-specific, or service-specific. A profile may include another profile as an input, enabling layered policy without merging mutable installation

prefixes. Whether a local layer overrides or rejects a site-wide collision is a profile policy, not a store invariant.

A process launched through Rax may hold a runtime lease on its closure. The lease prevents collection even if the profile that launched the process is switched or deleted while the process continues to run. Leases expire through an explicit protocol rather than by guessing from profile state.

Garbage Collection

There is no command that destructively uninstalls a store object. A user removes an output from a profile or deletes a retained generation. The garbage collector then computes reachability from:

- current and retained profile generations;
- explicit pins and administrative roots;
- active builds and transfers;
- runtime leases; and
- policy-defined archival roots.

Objects outside the transitive closure of those roots are collectible. A dry run must explain both sides of the decision: which objects would be deleted and which root keeps each retained object alive. Explainability is part of safe operation, not merely a user-interface convenience.

Collection removes physical objects only after removing no-longer-needed aliases and realization records according to a crash-safe order. A store verification command can reconstruct reachability, rehash objects, and detect broken aliases without trusting cached database state.

Substitution and Binary Caches

Binary deployment is an optimization of source deployment, not a separate package format. A cache indexes a PlanId, platform, and output name to a signed realization record and object archive. The client:

1. requests a realization for the exact PlanId;
2. applies local trust policy to the record and signer;
3. fetches the object and referenced closure;
4. computes each ObjectId locally; and
5. commits the verified realization through the normal store protocol.

Trust in a cache does not replace content verification. It answers a different question: whether the client accepts the claim that a particular ObjectId is an authorized realization of the PlanId. Rebuilding the Plan independently can provide stronger reproducibility evidence. Guix's work on authenticated updates and independently verifiable supply chains demonstrates why source history, build provenance, and binary attestation must be treated as connected but distinct concerns [11].

Cache selection, signature thresholds, organizational authorization, and retention are policy. The store protocol only defines the records and verification rules required to implement those policies.

Implementation in Raku and Raku++

The reference implementation is written in Raku and compiled with Raku++. Raku++ is a from-scratch Raku implementation written to the 2017 revision of C++ (C++17), with a hand-written lexer, parser, tree-walking evaluator, and native C++ code-generation mode [9]. Its current grammar and action support is sufficient for an external Rax parser, while deep parser replacement and slangs need not be part of the Rax design [9,10].

The front end uses a Raku grammar and an actions object. Actions immediately produce native immutable values and source spans; Match objects do not escape into the evaluator. The implementation favors Raku's native data model:

- Map, List, Pair, Set, and scalar values represent records and trees;
- enums identify node, step, input, and output kinds;
- subsets validate digests and identifiers;
- proto and multi routines implement evaluation, validation, canonical encoding, and store operations; and
- classes are reserved for grammar actions, exceptions, stateful resources, and objects with real lifecycle or identity.

The implementation pipeline is divided into independently testable modules:

Rax::Grammar	source -> Match
Rax::Actions	Match -> surface data
Rax::Eval	surface data -> values
Rax::Validate	values -> checked plan
Rax::IR	checked plan -> Plan IR
Rax::Canon	Plan IR -> canonical bytes
Rax::Digest	bytes -> PlanId or ObjectId
Rax::Store	immutable commit and query
Rax::Realize	authorized effects
Rax::Profile	generations and roots

The evaluator, canonical encoder, and Plan validator contain no network or process operations. Side effects enter through narrow adapters. The command-line program is orchestration, not the domain core.

Raku++ is the normative compiler for Rax. Tests compare interpreted Raku++ with a binary produced by its native compilation mode. The project does not require Rakudo parity or constrain its design around another implementation's behavior. When Rax exposes a Raku++ defect, the preferred remedy is a focused compiler regression and patch rather than a hidden compatibility workaround.

Validation Strategy

The first milestone proves the language boundary before attempting package builds. It must provide parsing, source-spanned diagnostics, strict evaluation, Plan IR, validation, canonical encoding, PlanId computation, and a native Rax executable.

Acceptance tests include:

- comments and whitespace do not change a PlanId;
- map insertion order does not change canonical bytes;

- a semantic value change does change the PlanId;
- repeated runs under the same pinned compiler produce identical output;
- interpreted and native-compiled Raku++ produce byte-identical plans;
- malformed source reports a file, line, column, and useful message; and
- evaluation cannot access the network, execute processes, or read arbitrary host paths.

Later milestones add store imports, structured realization, profiles, garbage collection, fixed-output fetchers, caches, and remote builders. Each milestone must finish a thin end-to-end slice before widening the surface.

Security as Evidence

A hash is a seal, not a conscience. It can show that bytes did not change. It cannot show that the bytes are kind, correct, recent, or wisely chosen. Rax therefore does not claim to solve software supply-chain security. No package manager can. Its narrower promise is useful enough: given an authenticated Plan and an explicit trust policy, Rax should make it independently verifiable that an installed object is an authorized realization of that exact Plan and its complete closure.

Trust does not disappear. It comes out of hiding.

Threat model. The adversary may submit unauthorized source, compromise a source repository or signing key, alter build parameters, operate a builder, cache, mirror, or transparency service, replay an old release, present different releases to different users, or poison a tool in the bootstrap chain. A builder may attempt undeclared file, process, clock, environment, or network access. A cache may bind the right name to the wrong bytes. Several actors may collude.

The initial model assumes that the selected cryptographic hash and signature algorithms hold, and that the local verifier, its policy state, and the machine executing that verifier have not already been subverted. Kernel, hardware, and cryptographic failure belong to a deeper model. Rax must state these assumptions instead of burying them beneath the word secure. The threat classes follow the source, build, distribution, selection, and usage boundaries described by Supply-chain Levels for Software Artifacts (SLSA) [13].

Security goals. Rax should bind an ObjectId to the exact PlanId that requested it; reject undeclared inputs and effects when the platform can enforce that claim; verify downloaded objects locally; preserve conflicting realizations; authenticate source and release history; resist rollback, freeze, and mix-and-match attacks; gather evidence from independent builders; and let the user decide which evidence is enough.

Non-goals. Rax cannot prove that source is benevolent, that a maintainer is honest, that reviewers understood the code, that a program is free of defects, or that several witnesses did not collude. It cannot make an unsafe configuration safe, recover a compromised machine, or prevent a network adversary from denying service. A reproducible backdoor remains a backdoor. SLSA makes similar limits explicit: provenance does not establish code quality, producer honesty, or the security of

every transitive dependency [13].

The realization evidence model. The store holds facts. Signatures hold claims. Policy makes a decision. These three must not be confused.

Fact: `hash(object bytes) = ObjectId`

Claim: `builder B says PlanId produced ObjectId`

Decision: `policy P accepts a set of facts and claims`

A realization record should contain the PlanId, ObjectId, declared inputs, observed references, granted capabilities, builder identity, platform, and the normalization rules applied to the output. The client verifies every fact it can verify locally. A signature proves who made a claim; it does not make the claim true.

Rax should export attestations compatible with SLSA provenance rather than invent a private vocabulary for common build facts [13]. Work outside the realizer, such as review, testing, release approval, and publication, can be expressed as signed steps in the style of in-toto [14]. The Rax Plan is stronger than a report written after the build: it is also the capability envelope that constrains the build while it runs.

Source and update authentication. Integrity begins before compilation. A perfect build of an unauthorized revision is still unauthorized. Rax package collections should carry authenticated source history, following the useful precedent established by Guix [11]. Release metadata must also say which keys or roles may authorize which objects, when that authority expires, and what version the client has already seen.

The Update Framework (TUF) supplies mature machinery for threshold keys, delegation, expiration, and resistance to rollback, freeze, fast-forward, and mix-and-match attacks [15]. Rax should adopt those semantics or a demonstrably equivalent protocol. A content hash says, "this is the old object." It does not say, "the old object is still the object you should run."

Independent and diverse rebuilding. Reproducible builds ask whether the same declared inputs produce the same bytes [12]. Agreement is valuable, but agreement is not independence. Two witnesses taught by the same compromised teacher are one witness wearing two coats.

Rax should distinguish repeated, independent, diverse, and bootstrapped builds. A second build on the same service is repeated. A build on another operator's machine is independent only in an administrative sense. A diverse build has a materially different toolchain or bootstrap ancestry. Diverse double-compiling (DDC) gives a researched method for testing whether a compiler binary corresponds to its claimed source under stated assumptions [18].

The compiler is not the only ancestor that matters. A 2026 preprint demonstrates a trusting-trust attack against the NixOS bootstrap using a malicious **strip** utility. The poisoned tool propagated through later generations, left the final dependency closure, and still altered most resulting executables [19]. The lesson is plain: any tool allowed to rewrite an artifact may carry the knife.

Bootstrap ancestry. Every builder attestation should identify the roots from which its result grew: the Raku++ ObjectId, the C++ compiler, assembler, linker, binary-rewriting

tools, runtime libraries, bootstrap seeds, operating system, sandbox implementation, and relevant hardware or virtual-machine class. This is a graph, not a badge.

Rax policy may then ask whether two agreeing builders actually descend from different roots. It may prefer minimized and auditable binary seeds, as Guix's bootstrap work does, but it must never call two builds diverse merely because their host-names differ [11,18,19].

Transparency and equivocation resistance. A signature can lie consistently to one victim. Transparency forces the lie into public view. Signed source revisions, realizations, reproduction results, and release authorizations should be eligible for inclusion in an append-only log. Clients verify inclusion; monitors verify that the log remains append-only; independent observers compare log views.

Sigstore's Rekor demonstrates a general transparency log for signed software-supply-chain metadata [16]. Nikitin et al. combine independent witnesses, verified builds, collective release authorization, and a tamper-evident release history [17]. Rax need not require one global log. A person, project, company, or nation may operate its own. What matters is that equivocation leaves evidence and that clients are not forced to trust the log operator in silence.

Local acceptance policy. Rax should carry evidence, not issue absolution. The final decision belongs to the user or organization. A policy may require:

- the downloaded bytes to match the ObjectId;
- a signed binding from the requested PlanId to that ObjectId;
- a builder whose isolation claims meet local requirements;
- fresh source and update authorization;
- agreement from a threshold of independent rebuilders;
- materially distinct bootstrap roots among those rebuilders; and
- inclusion of the release and its evidence in monitored logs.

Failure should explain which fact, claim, or policy rule was missing. The user should see *content verified*, *plan bound*, *isolated*, *reproduced*, *diverse*, or *transparent*, not a green shield that pretends all these words mean the same thing.

Residual trust. Rax cannot make evil source good, careless source wise, or a compromised kernel faithful. It cannot prove that reviewers read what they signed or that independent builders are not secretly one actor. It can make each remaining trust decision smaller, named, and challengeable.

That is the security claim: not trustlessness, but evidence. Not certainty, but a chain whose links can be held in the hand. The store tells us what is there. The evidence tells us who said it should be there. Local policy decides whether to open the door.

Related Work

Nix. Rax directly inherits Nix's separation of user and installation space, immutable isolated variants, closure deployment, atomic profiles, substitution, and reachability-based collection [1,2]. It differs principally at the language and identity

boundaries: Rax is strict by default, lowers to a closed data plan, represents build operations structurally, and separates plan identity from observed content identity.

Guix. Guix demonstrates the benefit of a rich host language, first-class package objects, functional package transformations, Scheme build programs, and explicit code staging [3,4]. Rax adopts the preference for language-native builders over shell text, but places package authors in a restricted external language rather than the full host language. This reduces library access and metaprogramming power in exchange for a smaller semantic and security surface.

Vesta. Vesta treated software construction as a functional computation and provided automatic dependency detection and repeatable builds [5]. Nix and Guix both cite Vesta as an important predecessor. Rax shares the goal of a precise computation graph but emphasizes deployment profiles and a portable user-space implementation.

Make and conventional build tools. Make expresses a dependency graph and incremental commands but traditionally relies on mutable working directories, timestamps, and ambient tools [6]. Rax can invoke such build systems inside a plan; it does not assume that their internal graph is sufficient to identify a portable deployment result.

Reproducible Builds. The Reproducible Builds project defines a build as reproducible when identical source, environment, instructions, and dependencies produce bit-for-bit identical artifacts [12]. Rax's PlanId identifies the declared side of that equation, while repeated ObjectIds provide direct observations of the result.

Open Questions

Several issues require implementation experience and measurement.

Recursive package collections. Strict evaluation must remain practical for large package sets. Selective imports, memoized module evaluation, explicit functions, and indexes may suffice. If not, Rax should add explicit delay rather than pervasive implicit laziness.

Self and dependency references. Logical plan addresses avoid a direct self-hash cycle, but they may reduce content sharing when embedded paths differ. The prototype must measure how often this matters and whether normalized reference tables or relocatable outputs are worth the added complexity.

Canonical tree rules. Platforms disagree about executable bits, symbolic links, extended attributes, sparse files, case folding, and path encodings. RAX-TREE/1 must define a portable core and explicit platform extensions.

Reference scanning. Byte scanning is general but can produce false positives and miss compressed or encoded references. Format-aware scanners can improve evidence but increase the trusted surface. Explicit declarations remain necessary.

Sandbox parity. Linux, Berkeley Software Distribution (BSD) systems, and macOS expose different isolation mechanisms. Capability claims must be testable and must not conceal

a weaker platform behind a common word such as hermetic.

Cyclic build graphs. The initial Plan graph should be acyclic. Bootstrapping cycles can be represented through staged seed objects or fixed points only after the ordinary model is stable.

Cache consensus. Multiple valid ObjectIds for one PlanId may indicate nondeterminism, platform leakage, or malicious substitution. User interfaces and policy engines must present this evidence without silently choosing a winner.

Implementation Roadmap

The proposed sequence is deliberately conservative.

- Milestone 0 Grammar, actions, source spans, evaluator, Plan IR, validation, canonical encoding, PlanId, command-line inspection, and native compilation.
- Milestone 1 Immutable local store, tree normalization, ObjectId, realization records, locking, and verification.
- Milestone 2 Structured Steps and one complete local C build from a fixed source.
- Milestone 3 Profiles, generations, atomic activation, rollback, roots, and leases.
- Milestone 4 Explainable mark-and-sweep garbage collection.
- Milestone 5 Fixed-output HTTPS, Git, and local-tree fetchers.
- Milestone 6 Signed realization records, object archives, binary caches, and local evidence policy.
- Milestone 7 Authenticated source and update history, transparency logs, independent and diverse rebuilding, and bootstrap-ancestry records.
- Milestone 8 Parallel scheduling, remote builders, cancellation, and log streaming.
- Milestone 9 A separate package collection and higher-level development environments.

The language must not absorb a feature merely because a future package collection might find it convenient. New syntax requires a demonstrated semantic need, a canonical representation, deterministic behavior, source-spanned diagnostics, and interpreter/native equivalence under Raku++.

Conclusion

Nix's lasting achievement was to make deployment resemble memory management: immutable values occupy isolated addresses, roots select live environments, and unreachable values can be collected [1,2]. Guix showed that richer language abstractions and staged build programs can make that model more extensible and more pleasant to program [3,4]. Rax seeks to preserve both lessons while drawing a firmer boundary between language evaluation and deployment effects.

A Rax expression evaluates strictly to a complete canonical Plan. A PlanId names the requested computation. Realization occurs only on demand. An ObjectId names the observed content. A realization record connects request to result. Profiles move atomically; garbage collection follows explicit

roots; caches supply verified realizations; and capabilities state what effects a builder may attempt.

Rax cannot certify goodness. It can require each accepted object to answer four questions: what Plan requested it, what bytes were produced, who witnessed the relation, and why local policy allowed it through. The aim is not to abolish trust, but to make every remaining trust decision visible and contestable.

The proposal is intentionally smaller than an operating-system distribution. Its first obligation is to prove that the language, identity, and store protocols are understandable and correct. If that foundation holds, package collections, development environments, services, and remote build systems can be layered above it without becoming hidden parts of the evaluator.

Rax therefore does not reject laziness as an implementation technique or a useful abstraction. It assigns laziness to the place where it yields the most operational value: deciding which work must be realized. Meaning itself remains strict, inspectable, and hashable.

References

- [1] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A Safe and Policy-Free System for Software Deployment," Proceedings of the 18th Large Installation System Administration Conference, USENIX Association, formerly the UNIX Users Group, 2004, pp. 79-92. <https://www.usenix.org/conference/lisa-04/nix-safe-and-policy-free-system-software-deployment>
- [2] E. Dolstra, E. Visser, and M. de Jonge, "Imposing a Memory Management Discipline on Software Deployment," Proceedings of the 26th International Conference on Software Engineering, Institute of Electrical and Electronics Engineers Computer Society, 2004, pp. 583-592.
- [3] L. Courtes, "Functional Package Management with Guix," European Lisp Symposium, 2013. arXiv:1305.4584. <https://arxiv.org/abs/1305.4584>
- [4] L. Courtes, "Code Staging in GNU Guix," Proceedings of the 16th Association for Computing Machinery Special Interest Group on Programming Languages International Conference on Generative Programming: Concepts and Experiences, 2017. Digital object identifier (DOI): 10.1145/3136040.3136045.
- [5] A. Heydon, R. Levin, T. Mann, and Y. Yu, "The Vesta Approach to Software Configuration Management," Compaq Systems Research Center Research Report 168, 2001.
- [6] S. I. Feldman, "Make - A Program for Maintaining Computer Programs," Software: Practice and Experience, vol. 9, no. 4, 1979, pp. 255-265.
- [7] The Raku Documentation Project, "Grammars" and "Grammar Tutorial," Raku language documentation. <https://docs.raku.org/language/grammars> https://docs.raku.org/language/grammar_tutorial
- [8] The Raku Documentation Project, "Functions: Multi-dispatch," Raku language documentation. <https://docs.raku.org/language/functions>
- [9] A. Shitov, "Raku++" and "Raku++ Architecture," project repository and architecture documentation, 2026. <https://github.com/ash/rakupp>
- [10] A. Shitov, "Raku++ Language-Mutation and Metaprogramming Coverage," project documentation, 2026. <https://github.com/ash/rakupp/blob/main/docs/METAPROGRAMMING.md>
- [11] L. Courtes, "Building a Secure Software Supply Chain with GNU Guix," arXiv:2206.14606, 2022. <https://arxiv.org/abs/2206.14606>
- [12] Reproducible Builds, "Definitions," project documentation. <https://reproducible-builds.org/docs/definition/>
- [13] Supply-chain Levels for Software Artifacts, "SLSA Specification," version 1.2, Linux Foundation, 2026. <https://slsa.dev/spec/v1.2/>
- [14] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes," Proceedings of the 28th USENIX Security Symposium, 2019, pp. 1393-1410. <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
- [15] The Update Framework, "The Update Framework Specification," version 1.0.35, project documentation. <https://theupdate-framework.github.io/specification/v1.0.35/>
- [16] Sigstore, "Rekor," project documentation. <https://docs.sigstore.dev/logging/overview/>
- [17] K. Nikitin, E. Kokoris-Kogias, P. Jovanovic, N. Gailly, L. Gasser, I. Khoffi, J. Cappos, and B. Ford, "CHAINIAC: Proactive Software-Update Transparency via Collectively Signed Skipchains and Verified Builds," Proceedings of the 26th USENIX Security Symposium, 2017, pp. 1271-1287.
- [18] D. A. Wheeler, "Countering Trusting Trust through Diverse Double-Compiling," Proceedings of the 21st Annual Computer Security Applications Conference, Institute of Electrical and Electronics Engineers Computer Society, 2005, pp. 28-40. DOI: 10.1109/CSAC.2005.17.
- [19] J. Malka, A. Sharma, M. Monperrus, S. Zacchiroli, and T. Zimmermann, "Trusting-Trust Attack against an Entire Linux Distribution through Binary Manipulation," arXiv:2607.24888, 2026. Preprint. <https://arxiv.org/abs/2607.24888>

Research Assistance Disclosure

The author conceived and directed the Rax design and assumes responsibility for its arguments, judgments, revisions, and final text. ChatGPT was used to assist with source discovery, citation planning, architectural discussion, and the preparation of an initial dictated draft under the author's direction. No claim or citation should be accepted without review against the referenced source.